

Expressions

Dr. Mohamed Yehia Dahab

Chapter 7 Topics

- Introduction
- Arithmetic Expressions
- Overloaded Operators
- Type Conversions
- Relational and Boolean Expressions
- Short-Circuit Evaluation
- Assignment Statements
- Mixed-Mode Assignment

Introduction

- Expressions are the fundamental means of specifying computations in a programming language
- To understand expression evaluation, need to be familiar with the orders of operator and operand evaluation
- Essence of imperative languages is dominant role of assignment statements

Arithmetic Expressions

- Arithmetic evaluation was one of the motivations for the development of the first programming languages
- Arithmetic expressions consist of operators, operands, parentheses, and function calls

Arithmetic Expressions: Design Issues

- Design issues for arithmetic expressions
 - Infix, prefix or postfix
 - operator precedence rules
 - operator associativity rules
 - order of operand evaluation
 - operand evaluation side effects
 - operator overloading
 - mode mixing expressions

Arithmetic Expressions: Operators

- A unary operator has one operand
- A binary operator has two operands
- A ternary operator has three operands

Arithmetic Expressions: Operator Precedence Rules

- The *operator precedence rules* for expression evaluation define the order in which “adjacent” operators of different precedence levels are evaluated
- Typical precedence levels
 - parentheses
 - unary operators
 - $**$ (if the language supports it)
 - $*$, $/$
 - $+$, $-$

Exceptions in Precedence Rules

- Most of programming languages give modulo operator “%” high precedence over unary operator “-”
 - That is to say $-a\%b$ is equivalent to $-(a\%b)$
 - $-8\%3 \rightarrow -(8\%3) \rightarrow -2$
 - **Perl** gives unary operator “-” a higher precedence
 - $X \text{ mod } Y = X - \lfloor x/y \rfloor * Y$

```
#!/usr/bin/perl  
$a= -5 % 3;  
print "Value of a is $a \n";
```


Infix, Prefix or Postfix

- In unary operator:
 - Prefix: $op\ a$
- In binary operator:
 - Infix: $a\ op\ b$
 - Prefix: $op\ a\ b$ **or** $op(a, b)$ **or** $(op\ a\ b)$
 - Postfix: $a\ b\ op$
- Most of programming languages use prefix for unary operators and infix in binary operators
- Lisp uses prefix for both operators and functions
 - $(*\ (+\ 3\ 2)\ 4)$; that like $(3+2)*4$ in infix
 - $(incf\ A\ 3)$

Arithmetic Expressions: Operator Associativity Rule

- The *operator associativity rules* for expression evaluation define the order in which adjacent operators with the same precedence level are evaluated
- Typical associativity rules
 - Left to right, except **, which is right to left
- APL is different; all operators have equal precedence and all operators associate right to left

Arithmetic Expressions: Operator Associativity Rule (Cont')

- ****** in Fortran and Ruby is right associative
 - $A^{**}B^{**}C \rightarrow A^{**}(B^{**}C)$
- **^** in Matlab is left associative
 - $A^B^C \rightarrow (A^B)^C$
- ****** in Ada is non-associative
 - $A^{**}B^{**}C$ *is illegal*
 - $(A^{**}B)^{**}C$ or $A^{**}(B^{**}C)$
- Precedence and associativity rules can be overridden with parentheses

Arithmetic Expressions: Operator Associativity Rule (Cont')

- Overflow possibility in
 - $A+B-C$
 - If A, B and C are large numbers
 - Right associative is safe while left associative is unsafe

Arithmetic Expressions (Ternary Operator): Conditional Expressions

- Conditional Expressions

- C-based languages (e.g., C, C++ and Java)

- An example:

- `average = (count == 0)? 0 : sum / count`

- Evaluates as if written like

- `if (count == 0) average = 0`
`else average = sum /count`

- Python

- `x = true_value if condition else false_value`*
`min= a if a<b else b`

Arithmetic Expressions: Operand Evaluation Order

- *Operand evaluation order*
 1. Variables: fetch the value from memory
 2. Constants: sometimes a fetch from memory; sometimes the constant is in the machine language instruction
 3. Parenthesized expressions: evaluate all operands and operators first (including the order of evaluation of function arguments in a function-call expression and the order of evaluation of the subexpressions within any expression)

Arithmetic Expressions: Potentials for Side Effects

- *Functional side effects*: when a function uses a non-local variable
- Problem with functional side effects:
 - It assumed that the following two statements are equivalent:
$$b = a + 10 + \text{fun}(a);$$
and
$$b = \text{fun}(a) + 10 + a;$$
 - When a function referenced in an expression alters another operand of the expression; e.g., for a parameter change:

Potentials for Side Effects (Cont')

```
#include <stdio.h>
int a = 10;
```

```
int fun(){
    a = 15;
    return 30;
}
```

```
int main(void) {
    // your code goes here
    int x = a + 10 + fun();
    a = 10;
    int y = fun() + 10 + a;
    printf(" x = %d \n y = %d \n", x, y);
    return 0;
}
```

x = 50

y = 55

<https://ideone.com/ZR8oQI>

Overloaded Operators

- Use of an operator for more than one purpose is called *operator overloading*
- Some are common (e.g., `+` for `int` and `float`)
- Some are potential trouble (e.g., `*` in C and C++)
 - Loss of compiler error detection (omission of an operand should be a detectable error)
 - Some loss of readability
 - Can be avoided by introduction of new symbols (e.g., Pascal's `div` for integer division and in Python `//`)

Overloaded Operators (continued)

- C++ and Ada allow user-defined overloaded operators
- Potential problems:
 - Users can define nonsense operations
 - Readability may suffer, even when the operators make sense

Type Conversions

- A *narrowing conversion* is one that converts an object to a type that cannot include all of the values of the original type e.g., `float` to `int`
- A *widening conversion* is one in which an object is converted to a type that can include at least approximations to all of the values of the original type e.g., `int` to `float`

Type Conversions: Mixed Mode

- A *mixed-mode expression* is one that has operands of different types
- A *coercion* is an implicit type conversion
- Disadvantage of coercions:
 - They decrease in the type error detection ability of the compiler
- In most languages, all numeric types are coerced in expressions, using widening conversions
- In Ada, there are virtually no coercions in expressions

Explicit Type Conversions

- Explicit Type Conversions
- Called *casting* in C-based language
- Examples
 - C: `(int) angle`
 - Ada: `Float (sum)`

Note that Ada's syntax is similar to function calls

Type Conversions: Errors in Expressions

- Causes
 - Inherent limitations of arithmetic
e.g., division by zero
 - Limitations of computer arithmetic e.g. overflow
 - Often ignored by the run-time system

Relational and Boolean Expressions

- Relational Expressions
 - Use relational operators and operands of various types
 - Evaluate to some Boolean representation
 - Operator symbols used vary somewhat among languages ($\neq$, $\neq$, $\neq$, $\neq$, $\neq$)

Relational and Boolean Expressions

- Boolean Expressions
 - Operands are Boolean and the result is Boolean
 - Example operators

FORTRAN 77	FORTRAN 90	C	Ada
<code>.AND.</code>	<code>and</code>	<code>&&</code>	<code>and</code>
<code>.OR.</code>	<code>or</code>	<code> </code>	<code>or</code>
<code>.NOT.</code>	<code>not</code>	<code>!</code>	<code>not</code>
			<code>xor</code>

Relational and Boolean Expressions: No Boolean Type in C

- C has no Boolean type--it uses `int` type with 0 for false and nonzero for true
- One odd characteristic of C's expressions: **a < b < c** is a legal expression, but the result is not what you might expect:
 - Left operator is evaluated, producing 0 or 1
 - The evaluation result is then compared with the third operand (i.e., `c`)

Relational and Boolean Expressions: Operator Precedence

- Precedence of C-based operators

prefix ++, --

unary +, -, prefix ++, --, !

*, /, %

binary +, -

<, >, <=, >=

=, !=

&&

||

Relational and Boolean Expressions: Operator Precedence (Cont')

- JavaScript and Php have two additional relational operators `===` and `!==` these are similar to `==` and `!=`, but prevent their operands from coercion
 - `"9" == 9` is true
 - `"9" === 9` is false
 - `x !== y;    ➔  !(x === y);`

Short Circuit Evaluation (continued)

- C, C++, and Java: use short-circuit evaluation for the usual Boolean operators (`&&` and `||`), but also provide bitwise Boolean operators that are not short circuit (`&` and `|`)
- Ada: programmer can specify either (short-circuit is specified with `and then` and `or else`)
- Short-circuit evaluation exposes the potential problem of side effects in expressions
e.g. `(a > b) || (b++ / 3)`

Assignment Statements

- The general syntax
`<target_var> <assign_operator> <expression>`
- The assignment operator
 - = FORTRAN, BASIC, PL/I, C, C++, Java
 - := ALGOLs, Pascal, Ada
- = can be bad when it is overloaded for the relational operator for equality

Assignment Statements: Conditional Targets

- Conditional targets (C, C++, and Java)
`(flag)? total : subtotal = 0`

Which is equivalent to

```
if (flag)
    total = 0
else
    subtotal = 0
```

Assignment Statements: Compound Operators

- A shorthand method of specifying a commonly needed form of assignment
- Introduced in ALGOL; adopted by C
- Example

`a = a + b`

is written as

`a += b`

Assignment Statements: Unary Assignment Operators

- Unary assignment operators in C-based languages combine increment and decrement operations with assignment

- Examples

`sum = ++count` (count incremented, assigned to sum)

`sum = count++` (count assigned to sum, count incremented)

`count++` (count incremented)

Assignment as an Expression

- In C, C++, and Java, the assignment statement produces a result and can be used as operands
- An example:

```
while ((ch = getchar()) != EOF) {...}
```

`ch = getchar()` is carried out; the result (assigned to `ch`) is used as a conditional value for the `while` statement

Assignment Statements (Cont')

- C based, Java and Python
 - `a = b = c = 4`
- Python
 - `x,y,z= 3,4,5`
 - `print(z,y,x)`

Mixed-Mode Assignment

- Assignment statements can also be mixed-mode, for example

```
int a, b;  
float c;  
c = a / b;
```

<https://ideone.com/ifyD6U>

- In Pascal, integer variables can be assigned to real variables, but real variables cannot be assigned to integers
- In Java, only widening assignment coercions are done
- In Ada, there is no assignment coercion

Summary

- Expressions
- Operator precedence and associativity
- Operator overloading
- Mixed-type expressions
- Various forms of assignment